UD 1.7. Interoperabilidad de Datos y Control de Versiones. XML-JSON....2
Guía de Referencia JSON....2
1. ¿Qué es JSON y por qué cambiamos XML?....2
2. Sintaxis: Las Reglas del Juego....2
A. Los Contenedores....2
B. Pares Clave/Valor....2
3. Tipado de Datos: La parte crítica de la tarea....3
🟢 1. Cadenas de Texto (String)....3
🔵 2. Números (Number)....3
🟠 3. Booleanos (Boolean)....3
🟣 4. Nulo (Null)....3
4. Guía de Traducción: De XML a JSON....4
Caso 1: Un elemento simple con atributos....4
Caso 2: Listas de elementos (El Array)....4
5. Errores Comunes (Checklist antes de entregar)....5
🛠️ Herramientas de Validación....5
Instrucciones de Realización....6
Archivos adjuntos....6
Archivo: legado_sucio.xml....6
Rúbrica de Evaluación....7

UD 1.7. Interoperabilidad de Datos y Control de Versiones. XML-JSON

Guía de Referencia JSON

1. ¿Qué es JSON y por qué cambiamos XML?

Hasta ahora hemos trabajado con XML, un lenguaje de marcado excelente para definir documentos con estructura estricta. Sin embargo, en el desarrollo web moderno y el intercambio de datos entre servidores y aplicaciones (APIs), necesitamos algo más ligero y rápido de procesar para JavaScript.

JSON (JavaScript Object Notation) es un formato de texto sencillo para el intercambio de datos.

- Es autodescriptivo: Fácil de leer y escribir para humanos.
- Es ligero: Ocupa menos espacio que XML (no tiene etiquetas de cierre).
- Es nativo: Se integra automáticamente con JavaScript, el lenguaje de la web.

Referencia Oficial:

Para profundizar en cualquier aspecto, utilizaremos la documentación de W3Schools, que será nuestra "Biblia" técnica para esta unidad: W3Schools - JSON Introduction

2. Sintaxis: Las Reglas del Juego

A diferencia de XML, donde usamos etiquetas <apertura> y </cierre>, en JSON jugamos con dos estructuras principales. Según W3Schools, la sintaxis deriva de la notación de objetos de JavaScript:

A. Los Contenedores

1. Objetos { ... }: Representan una entidad única (un libro, un alumno, un coche). Se definen con llaves.
2. Arrays [...]: Representan listas o colecciones de cosas (varios libros, notas, inventario). Se definen con corchetes.

B. Pares Clave/Valor

Los datos siempre se escriben en pares campo : valor.

- La Clave (nombre del campo) SIEMPRE debe ir entre comillas dobles.
- El Valor depende del tipo de dato (veremos esto abajo).

Ejemplo básico:

JSON

```
{
  "nombre": "Ana",
  "edad": 25
}
```

3. Tipado de Datos: La parte crítica de la tarea

En XML, todo el contenido es texto.

- <precio>25.50</precio> --> Para el ordenador es texto.
- <disponible>true</disponible> --> Para el ordenador es texto.

En JSON, los tipos de datos importan. Para realizar correctamente la práctica, debéis respetar rigurosamente estas reglas (Consultar: JSON Data Types en W3Schools):

🟢 1. Cadenas de Texto (String)

Se usan para nombres, descripciones, fechas o códigos alfanuméricos.

- Regla: SIEMPRE entre comillas dobles.
- ✅ "nombre": "Juan"
- ❌ "nombre": Juan (Error de sintaxis)

🔵 2. Números (Number)

Se usan para precios, edad, stock, coordenadas, ID numéricos.

- Regla: NUNCA llevan comillas. Si pones comillas, lo conviertes en texto.
- ✅ "precio": 19.99 (Correcto: se puede operar matemáticamente).
- ✅ "año": 2025
- ⚠ "precio": "19.99" (Incorrecto: es una cadena de texto, no un número).

🟠 3. Booleanos (Boolean)

Se usan para estados binarios: Sí/No, Verdadero/Falso, Activo/Inactivo.

- Regla: Se escriben en minúscula y SIN comillas.
- ✅ "enStock": true
- ✅ "esNovedad": false
- ❌ "enStock": "true" (Mal: esto es un texto "true").

🟣 4. Nulo (Null)

Se usa cuando un campo está vacío o no tiene valor.

- ✅ "comentarios": null

4. Guía de Traducción: De XML a JSON

Para vuestra tarea de migración, seguid este patrón visual para transformar vuestros archivos.

Caso 1: Un elemento simple con atributos

XML Original:

XML

```
<producto id="501" moneda="EUR">
  <nombre>Teclado Mecánico</nombre>
  <precio>45.00</precio>
  <oferta>si</oferta> </producto>
```

JSON Resultante (Transformación Correcta):

Observad cómo los atributos (id, moneda) pasan a ser propiedades normales dentro del objeto, y cómo cambiamos los tipos de datos.

JSON

```
{
 "id": 501,
 "moneda": "EUR",
 "nombre": "Teclado Mecánico",
 "precio": 45.00,
 "enOferta": true
}
```

Nota: Hemos cambiado el texto "si" por el booleano true y el "501" a número.

Caso 2: Listas de elementos (El Array)

Cuando en XML tenéis etiquetas repetidas, en JSON obligatoriamente necesitáis un Array [].

XML Original:

XML

```
<biblioteca>
  <libro>...</libro>
  <libro>...</libro>
  <libro>...</libro>
</biblioteca>
```

JSON Resultante:

JSON

```
{
 "biblioteca": {
  "libros": [
   { "titulo": "Libro A", "precio": 10 },
   { "titulo": "Libro B", "precio": 15 },
   { "titulo": "Libro C", "precio": 12 }
  ]
 }
}
```

5. Errores Comunes (Checklist antes de entregar)

Antes de subir vuestra práctica a Moodle/Git, verificad esto:

1. La "Coma Traicionera": En JSON, el último elemento de una lista u objeto NO lleva coma al final.
    - *Mal:* {"a": 1, "b": 2,}
    - *Bien:* {"a": 1, "b": 2}
2. Comillas en las Claves: Todas las claves deben tener comillas.
    - *Mal:* { edad: 20 }
    - *Bien:* { "edad": 20 }
3. Tipado falso: Revisad que no habéis puesto comillas a los números ("20") ni a los booleanos ("true").

🛠️ Herramientas de Validación

No confiéis solo en vuestra vista. Usad herramientas para verificar que vuestro código es válido:

- JSONLint - The JSON Validator
- Extensión de VS Code: "JSON Tools" o el formateador por defecto (Click derecho > Format Document).

Descripción de la tarea:

"La aplicación de vuestro Catálogo Web sigue creciendo. El equipo de *backend* necesita consumir vuestros datos mediante una API REST, por lo que el formato XML ha quedado obsoleto para la transmisión de datos.

Objetivo: Migrar vuestro propio catálogo y un sistema legado antiguo a formato **JSON**, organizando el proyecto profesionalmente mediante un repositorio **Git**."

Instrucciones de Realización

Paso 1: Preparación del Entorno (Git)

1. Crea una carpeta local llamada UD1.7
2. Inicializa un repositorio local (git init).
3. Crea la siguiente estructura de directorios:

   Plaintext

   ```
   /data
     ├── /xml   (Aquí guardarás los originales)
     └── /json  (Aquí guardarás los archivos migrados)
   README.md    (Breve descripción de la práctica)
   ```

Paso 2: Migración del Catálogo Propio (Sesiones 3 y 5 de dic)

1. Recupera el archivo catalogo.xml que creaste en la práctica anterior y copialo en la carpeta /xml.
2. Crea el archivo /json/catalogo.json.
3. Traduce tus datos a JSON, aplicando **tipado estricto**:
   - Precios y cantidades -> Number (sin comillas).
   - Disponibilidad/Stock --> Boolean (true/false).
   - Listas de productos --> Array [...].

Paso 3: El Reto "Legacy" (Sesión 10 de dic)

1. Descarga el archivo legado_sucio.xml (adjunto en la tarea).
2. Analiza su estructura (contiene errores y mezcla atributos con etiquetas).
3. Genera un archivo limpio /json/inventario_legacy.json que resuelva la estructura compleja.

Paso 4: Entrega

- Realiza *commits* lógicos conforme avances en el trabajo.
- **Para entregar:** Sube tu repositorio a GitHub/GitLab y pega el enlace, O BIEN comprime la carpeta completa (incluyendo la carpeta oculta .git) si no usas servidor remoto.

Archivos adjuntos

Archivo: legado_sucio.xml

XML

```
<?xml version="1.0"?>
<sistema_inventario>
```

```
  <zona_almacen codigo="A1">
    <estante nivel="3">
      <item ref="X-99" estado="roto">Monitor CRT</item>
      <item ref="X-100" estado="nuevo">Teclado Mecánico</item>
    </estante>
    <responsable>
      <nombre>Juan</nombre>
      <permisos>admin</permisos>
      <permisos>editor</permisos>
    </responsable>
  </zona_almacen>
</sistema_inventario>
```

Rúbrica de Evaluación

Criterio: Calidad de la migración de datos y gestión del proyecto

Calificación	Nivel de Logro	Detalles del criterio
0 Puntos	**No Evaluable**	No se entrega la tarea o los archivos no son legibles.
2.5 Puntos	**Sintaxis Incorrecta**	Entrega los archivos JSON, pero contienen errores de sintaxis que impiden su validación (comas sobrantes, claves sin comillas, etc.). No se respeta la jerarquía original.
5.0 Puntos	**JSON Válido (Básico)**	Consigue migrar su catálogo y el ejercicio legacy a JSON válido. Sin embargo, **todo son Strings** (no diferencia números ni booleanos) o **no utiliza Git** (entrega archivos sueltos sin estructura de directorios).
7.5 Puntos	**JSON Tipado Correcto**	La migración a JSON es perfecta: usa arrays, objetos, números y booleanos correctamente en ambos ejercicios. La estructura de datos es limpia. **Pero:** La entrega no sigue la estructura de directorios pedida o no incluye el historial de Git (entrega plana).
10 Puntos	**Entrega Profesional (Git)**	**Todo lo anterior perfecto + Control de Versiones.** 1. JSONs validados y tipados. 2. Estructura de carpetas /xml y /json correcta. 3. El repositorio Git está inicializado correctamente y tiene al menos 3 commits reflejando el progreso del trabajo.